



ANSWERED



CORRECT



INCORRECT



FLAGGED

0 / 70

0%

SUBMIT ALL & SCORE

REVEAL ALL ANSWERS

RESET EXAM

PRINT / SAVE PDF

NEXT UNANSWERED ↓

ALL (70)

COMPOSITION (16)

CONTENT AGGREGATION (7)

CUSTOMIZING USD (4)

DATA EXCHANGE (11)

DATA MODELING (9)

DEBUGGING (8)

PIPELINE DEV (10)

VISUALIZATION (5)

COMPOSITION

#1

COMPOSITION · 23% EXAM WEIGHT

A studio uses a ``/_class_Shadeable`` class prim to broadcast a render-visibility override to all assets that inherit from it. An individual asset's local layer sets ``visibility = invisible`` on its root prim. Which value wins?

A

The class prim's value wins because inherits arcs define the authoritative look

B

The asset's local opinion wins, because in LIVERPS local opinions (L) are stronger than inherits (I)

C

The two opinions are averaged — the asset is rendered at 50% opacity

D

USD raises a composition conflict and neither value is used

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: LIVERPS: **Local (L) > Inherits (I)**. A local opinion on the asset's own root prim is always stronger than what flows in from a class prim via ``inherits``. The asset's ``visibility =`



invisible` wins. This is why class prims are good for 'broad defaults' — individual assets can always override them locally.

#2

COMPOSITION · 23% EXAM WEIGHT

In USD, what does the term 'opinion' mean in the context of scene description?

A

A user preference stored in the application settings that affects USD display

B

A value for an attribute or metadata that a specific layer is asserting — one layer's contribution to the final composed value

C

A render setting that tells the renderer how to interpret geometry

D

A validation result from `usdchecker` indicating a possible compliance issue

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: In USD, an **opinion** is the value that a specific layer asserts for an attribute, property, or metadata entry. Multiple layers can have opinions on the same attribute; the composition system uses LIVERPS to determine which opinion wins. The winning opinion becomes the *resolved value* on the composed stage.

#3

COMPOSITION · 23% EXAM WEIGHT

A reference arc and a payload arc both point to the same layer file. An artist unloads the prim that uses the payload arc. What happens to the prim that uses the reference arc?

A

The reference prim is also unloaded because both arcs point to the same file

B

The reference prim remains fully loaded — reference arcs are not affected by payload load/unload operations

C

Both prims enter a 'half-loaded' state sharing whatever data is currently cached

D

USD raises an error because the same file cannot be used by both a reference and a payload

[CHECK ANSWER](#)[SHOW EXPLANATION](#)[FLAG](#)

Explanation: Reference arcs and payload arcs are independent. Unloading a payload prim only affects that prim's payload arc. A separate prim that references the same file via a reference arc is not affected — references are always loaded. This is fundamental to the difference between the two arcs: payloads are controllable; references are not.

#4

COMPOSITION • 23% EXAM WEIGHT

A scene layer defines `/World/Tree` as an `over`. No other layer defines `/World/Tree` with a `def`. What is the result on the composed stage?

- ☒ A The prim exists as a typeless prim with whatever opinions the `over` contributed
- ☐ B The `over` is silently ignored — USD only composes prims that have at least one `def` somewhere in the layer stack
- ☐ C The `over` is treated as a `def` and the prim is created with no type
- ☐ D USD raises a fatal error because `over` requires an existing definition

[CHECK ANSWER](#)[SHOW EXPLANATION](#)[FLAG](#)

Explanation: A prim defined only by `over` (with no `def` anywhere in the layer stack) is called a **typeless def** or **pure over**. It does exist on the composed stage as a typeless prim and its opinions are accessible. It will not have an `IsA` schema type. Some traversal predicates exclude typeless prims, but they are valid scene description. This is a common source of unexpected behavior when `def` layers are missing.

#5

COMPOSITION • 23% EXAM WEIGHT

What is the purpose of the `edit target` in a USD stage?

- ☐ A It specifies which renderer should be used for the current viewport session
- ☒ B It designates which layer (and optionally which prim path mapping) receives newly authored opinions when a stage edit API is called

C It sets the default time code for all attribute queries on the stage

D It defines the root layer that all sublayers are resolved relative to

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: The **edit target** determines where new scene description is written when you call USD's authoring APIs (e.g., ``UsdAttribute::Set()``, ``UsdPrim::SetActive()``). By default it is the root layer, but you can redirect edits to any layer in the stage's layer stack — including sublayers, layers inside references, or even variant layers. This is how artists and tools author opinions into specific layers.

#6

COMPOSITION • 23% EXAM WEIGHT

A value clip is configured on a prim. The clip's ``clipActive`` metadata specifies that clip 0 is active from stage time 1 to 50, and clip 1 is active from stage time 51 to 100. What happens when the stage is queried at time 75?

A USD uses clip 0 because it was defined first

B USD uses clip 1, which is active for the time range covering stage time 75

C USD interpolates between clip 0 and clip 1 to produce a blended value

D USD falls back to the attribute's default value when multiple clips are defined

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Value clips use ``clipActive`` to define which external animation clip is active at each stage time. At time 75, ``clipActive`` specifies **clip 1** is active (range 51–100). USD reads the attribute value from clip 1's layer at the corresponding local clip time. There is no blending between clips — the active clip is used exclusively during its active time range.

#7

COMPOSITION • 23% EXAM WEIGHT

A USD ``over`` prim in a shot layer adds a new child prim ``Sparkle`` under a referenced asset's existing prim. The referenced asset does not define

`Sparkle`. After composition, what is the status of `/World/Prop/Sparkle`?

- ☐ A It is silently ignored — you cannot add child prims to referenced assets from a downstream layer
- ☒ B It exists on the composed stage as a new prim contributed by the shot layer's local opinion, sitting alongside the referenced asset's prims
- ☐ C It causes a composition error because the referenced asset does not define that path
- ☐ D It is added to the referenced asset file on disk automatically

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: USD's composition model allows downstream layers to **add new prims** (as children of any existing prim, even one from a reference) via local opinions. The shot layer's `over` on `/World/Prop` can contain a child `def Sparkle` that creates a new prim at `/World/Prop/Sparkle` on the composed stage. The referenced asset is not modified — this is purely a local addition in the stronger layer.

#8

COMPOSITION • 23% EXAM WEIGHT

Which statement correctly describes the relationship between `sublayers` and `references` with respect to namespace?

- ☐ A Both sublayers and references graft content under a target prim path
- ☒ B Sublayers contribute their prims at the same namespace root as the stage; references graft content under the specific prim path that holds the reference arc
- ☐ C Sublayers are limited to contributing material prims; references can contribute any prim type
- ☐ D References share the same namespace as sublayers — both are merged at the root level

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Sublayers merge their entire prim namespace at the stage root level — a prim at `/World/Chair` in a sublayer appears at `/World/Chair` in the composed stage. **References** graft

content under a specific prim — a file with defaultPrim `Chair` referenced at `/Scene/Furniture` creates `/Scene/Furniture/Chair` (and descendants) in the composed stage. This namespace behavior is a fundamental architectural difference.

#9

COMPOSITION · 23% EXAM WEIGHT

A pipeline engineer wants to use the `specializes` arc to share a look-dev base for all metal assets. Why is `specializes` preferred over `inherits` for this use case?

A

Specializes supports time-sampled opinions; inherits does not

B

Specializes arcs are weaker than all other arcs including local opinions, meaning asset-specific look refinements always override the base while still receiving the base's defaults — the base 'specializes' the general concept for a specific case

C

Inherits can only be used with class prims; specializes works with any prim type

D

Specializes is more memory-efficient than inherits for large numbers of prims

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Specializes is the *weakest* composition arc in LIVERPS. This means: the specializing asset's own opinions (local, inherits, variants, references, payloads) all override the base's opinions — the base provides sensible defaults while specific assets can freely refine them. `Inherits` is stronger than variants/references, which can interfere with asset-level overrides. Specializes is ideal for 'base class with refinements' patterns in look-dev.

#10

COMPOSITION · 23% EXAM WEIGHT

A shot pipeline wants to isolate a specific prim from all camera framing and rendering tools without deleting it. Which USD mechanism accomplishes this without modifying the asset file?

A

Set `purpose = guide` on the prim in the shot layer so it is excluded from render and proxy traversal

B

Set `active = false` on the prim in the shot layer — deactivated prims are pruned from traversal and invisible to renderers

C Add the prim to a `_class_Hidden` class and override its `visibility` attribute to `invisible`

D Both A and B are valid approaches with different semantic meanings

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Both approaches work but have different semantics: `active = false` completely *deactivates* the prim — it is pruned from all traversal and acts as if it doesn't exist. `purpose = guide` keeps the prim traversable but signals that it is a non-renderable guide element excluded from render/proxy traversal. The right choice depends on whether you want the prim accessible to tools at all or just excluded from rendering.

#11

COMPOSITION • 23% EXAM WEIGHT

A USD layer has `subLayers = [ @./lighting.usda@, @./animation.usda@, @./layout.usda@ ]`. A developer changes this to `subLayers = [ @./animation.usda@, @./lighting.usda@, @./layout.usda@ ]`. What changes in the composed stage?

A Nothing changes — sublayer order only affects load performance, not opinion strength

B Attributes where `animation.usda` and `lighting.usda` have conflicting opinions will now resolve to `animation.usda`'s value instead of `lighting.usda`'s

C All opinions from `lighting.usda` are now ignored because it is no longer first

D The stage must be fully reloaded before any ordering change takes effect

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Sublayer order determines opinion strength: earlier layers are stronger. By moving `animation.usda` before `lighting.usda`, animation's opinions now win over lighting's where they conflict. For example, if both layers set `xformOp:translate` on the same prim, the composed value changes from `lighting.usda`'s value to `animation.usda`'s value. **Sublayer ordering is a critical pipeline design decision.**

COMPOSITION • 23% EXAM WEIGHT

#12

An asset at `vehicle.usda` is referenced twice into the same scene: once at `/Scene/CarA` and once at `/Scene/CarB`. Both use `instanceable = true` and reference the same file. A variant `color = red` is set on `CarA` and `color = blue` on `CarB`. Can they still share an instancing prototype?

A

Yes — both prims share the same prototype regardless of variant selections

B

No — USD creates separate prototypes for each unique combination of composition metadata, including variant selections, so CarA and CarB get different prototypes

C

Yes — variant selections are ignored when `instanceable = true` is set

D

No — USD does not support variant sets on instanceable prims

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: USD creates a prototype for each unique combination of **composition metadata** (reference paths, variant selections, etc.). Since `CarA` selects `color = red` and `CarB` selects `color = blue`, they have different composition fingerprints and **cannot share the same prototype** — USD creates two separate prototypes. Variant selections directly affect instancing prototype sharing.

#13

COMPOSITION • 23% EXAM WEIGHT

A pipeline architect reviews a USD asset that has both `prepend references` and `prepend payload` arcs on the same prim. According to LIVERPS, which arc's opinions are stronger?

A

The reference arc, because References (E) are stronger than Payloads (P) in LIVERPS

B

The payload arc, because payloads load after references and their data is more recent

C

Both arcs have equal strength since they are authored in the same layer

D

The arc listed first in the file has stronger opinions

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: In LIVERPS, **Externally referenced (E/References)** are stronger than **Payloads (P)**. When the same prim has opinions from both a reference arc and a payload arc, the reference arc's opinions win. This ordering is why the reference/payload split pattern works: the reference layer defines the asset interface (stronger), and the payload provides geometry data (weaker, overridable by the reference).

#14

COMPOSITION • 23% EXAM WEIGHT

A `list editing` operation using `delete` is applied to a reference list. What does this accomplish?

A

It removes the referenced file from disk

B

It removes a specific reference target from the composed reference list, preventing that arc from being used in composition without modifying the layer that originally added it

C

It marks the reference as inactive but keeps it in the list for future use

D

It replaces all references in the list with the specified deleted item's default prim

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: USD's **list editing** allows a stronger layer to `delete` items from a weaker layer's list (references, payloads, inherits, etc.) without modifying the weaker layer. The `delete` operation removes the specified arc from the composed list, preventing it from contributing to composition. This enables a downstream layer to 'remove' a reference that was added by a weaker layer—a key non-destructive operation.

#15

COMPOSITION • 23% EXAM WEIGHT

A character asset has a `lod` variant set. In a performance-critical shot, the pipeline wants all instances of this character to use `lod = low` regardless of what other stages have selected. Where should this override be authored for maximum strength?

A

Inside the character asset file itself, changing the default variant selection

B In the shot's root layer as a local opinion, which has stronger priority than any sublayer or reference-level selection

C In the set-dressing layer that places the characters

D In a payload arc above the character reference

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: The **shot's root layer** is at the top of the sublayer stack and thus has the strongest local opinion. Authoring the variant selection ``lod = low`` as a local opinion in the root layer (the edit target) ensures it overrides any selection made in weaker sublayers, references, or the asset's own default. For maximum strength, author overrides as local opinions in the strongest layer of the stack.

#16

COMPOSITION • 23% EXAM WEIGHT

After composing a stage, an artist discovers a prim has ``double3 xformOp:translate = (0, 0, 0)`` even though they expected ``(5, 0, 0)``. They check the asset file and find ``(5, 0, 0)`` is correctly authored there. What should they check next to debug this?

A Whether the USDC file is corrupted and needs to be re-exported

B Whether a stronger layer (a sublayer above the asset, or a reference with a stronger arc) is contributing a ``(0, 0, 0)`` opinion that overrides the asset's value

C Whether the asset uses USDA format — USDA does not support translate values above (1, 1, 1)

D Whether the prim's ``kind`` metadata is set incorrectly

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: When an expected value is overridden on the composed stage, the first investigation step is to find whether a **stronger layer has a conflicting opinion**. Use ``UsdProperty::GetPropertyStack()`` or the usdview LayerStack tab to see all layers contributing a

`xformOp:translate` opinion and their order. A layer above (stronger than) the asset file with `(0, 0, 0)` is the almost certain culprit.

CONTENT AGGREGATION

#17

CONTENT AGGREGATION · 10% EXAM WEIGHT

A scene uses native USD instancing with `instanceable = true` on 500 tree prims. A pipeline tool tries to access `/Forest/Tree\_042/Branch/Leaf` directly. What will it find?

A

The actual prim with all its authored attributes accessible for editing

B

An instance proxy — a read-only virtual representation of that prim through the shared prototype

C

Nothing — child prims inside instanceable prims are hidden from all API access

D

A copy of the prim that can be edited independently without affecting other instances

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Accessing a child prim within an instanceable prim returns an **instance proxy** — a virtual, read-only view of that prim as seen through the shared prototype. Instance proxies allow you to query the prim's properties and relationships but restrict authoring (since the prototype is shared). To traverse instance proxies, `UsdPrimRange` must be configured with `UsdTraverseInstanceProxies`.

#18

CONTENT AGGREGATION · 10% EXAM WEIGHT

Which `kind` value in the USD model hierarchy acts as a container for organizing prims that are not themselves models, but groups them for scene management purposes?

A

assembly — a collection of other models

B

group — a container for organizing models within the hierarchy without itself being a leaf model

C subcomponent — an internal part of a component

D scope — a lightweight organizational prim with no transform

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: `group` is the model kind for an organizational container of other models. Unlike `assembly` (which is a model in its own right), `group` is used purely for hierarchical organization — grouping components or assemblies together. Crucially, a `group` does not break the model hierarchy traversal — tools that look for models can still find them inside a `group`.

#19

CONTENT AGGREGATION • 10% EXAM WEIGHT

A set-dressing tool is aggregating a city scene with buildings, vehicles, and street furniture — all as component assets. The artist wants the entire city block to be treatable as a single referenceable unit. Which kind should be assigned to the city block's root prim?

A component — because the city block is the indivisible unit of the scene

B subcomponent — because the buildings and vehicles are subcomponents of the block

C assembly — because the city block is a collection of component assets

D group — because the city block groups the components without being a model itself

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: An **assembly** is the correct kind for an aggregation of component assets that itself needs to be referenceable as a unit. The city block contains components (buildings, cars, furniture) and is itself a model that can be referenced into larger scenes. `Component` would be wrong — it implies a leaf-level, non-aggregating asset. `Assembly` correctly describes the hierarchical aggregation.

#20

CONTENT AGGREGATION • 10% EXAM WEIGHT

A `UsdGeomPointInstancer` defines positions and orientations for 10,000 grass blades. The artist wants to apply wind animation that moves all grass uniformly. What is the most efficient approach?

- A Animate the `positions` array on the `PointInstancer` with 10,000 new positions per frame
- B Add time-sampled `velocities` to the `PointInstancer` so the renderer can compute per-instance motion
- C Animate the prototype prim's transform — since all instances share the prototype, the animated transform propagates to all instances**
- D Create a separate `PointInstancer` per frame of animation

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Animating the **prototype prim's transform** is the most efficient approach for uniform motion. Since all instances share the same prototype, any animation on the prototype affects all instances simultaneously. This requires only one set of time samples (on the prototype) rather than updating 10,000 position entries per frame. For non-uniform wind (each blade moving differently), per-instance `positions` would be needed.

#21

CONTENT AGGREGATION • 10% EXAM WEIGHT



A pipeline is debating whether to use USD native instancing or `UsdGeomPointInstancer` for a library of 20 unique hero props scattered around a scene (each placed 2–5 times). Which is more appropriate?

- A `UsdGeomPointInstancer` — it is always faster for any repeated geometry
- B Native instancing with `instanceable = true` — for a small number of unique assets each placed a handful of times, native instancing's per-prim flexibility and simpler composition are more appropriate**
- C Neither — 20 unique assets should never be instanced in USD
- D `UsdGeomPointInstancer` — it is required any time the same asset appears more than once

[CHECK ANSWER](#)[SHOW EXPLANATION](#)[FLAG](#)

Explanation: Native instancing is more appropriate for small-scale repetition of unique assets (20 props, 2–5 copies each). It preserves full composition flexibility per instance and works well with USD's scenegraph traversal. `UsdGeomPointInstancer` shines for *massive* repetition (thousands to millions of copies) where individual prim overhead is prohibitive. Choosing the right instancing style for the scale of repetition is a key design decision.

#22

CONTENT AGGREGATION • 10% EXAM WEIGHT



A modular building system in USD uses multiple component assets (walls, floors, roofs) referenced together under an assembly prim. A downstream stage needs to swap just one wall variant. Which approach preserves the assembly structure while enabling the per-component override?

A

Flatten the assembly and edit the wall geometry directly in the shot layer

B

In the shot layer, author an `over` on the wall component prim within the assembly, setting the desired variant selection — the assembly structure and other components remain unchanged

C

Remove the assembly reference and re-reference each component separately with the new wall variant

D

Create a new variant set on the assembly level that includes the wall variant as a nested option

[CHECK ANSWER](#)[SHOW EXPLANATION](#)[FLAG](#)

Explanation: The correct approach is to use an **`over`** in the shot layer targeting the specific wall component prim inside the assembly, and author the variant selection there. This is a non-destructive, local override that leaves the assembly structure, other components, and the original asset files completely unchanged. It demonstrates USD's composability — downstream stages can selectively override specific parts of complex assembled hierarchies.

#23

CONTENT AGGREGATION · 10% EXAM WEIGHT



A scene has 50,000 particles represented as a `UsdGeomPointInstancer`. The particles are hidden during the first 50 frames. What is the most efficient way to hide all particles at once for frames 1–50?

- ☒ A Set `visibility = invisible` on the `PointInstancer` prim with a time sample at frame 1, then `visibility = inherited` at frame 51
- ☐ B Add all 50,000 instance indices to `invisibleIds` for frames 1–50
- ☐ C Set `active = false` on the `PointInstancer` prim for frames 1–50
- ☐ D Delete the `PointInstancer` prim in frame 1 and recreate it at frame 51

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Setting `visibility = invisible` with a time sample on the `PointInstancer` prim itself is the most efficient approach. It hides the entire instancer (all 50,000 instances) with a single attribute time sample. The alternative — populating `invisibleIds` with 50,000 entries — is far more expensive in memory and I/O. Setting `active = false` would also work but has broader semantic implications (removes the prim from traversal entirely).

CUSTOMIZING USD

#24

CUSTOMIZING USD · 6% EXAM WEIGHT



A studio needs to add a `simulation:restLength` attribute to any prim type in the scene for cloth simulation. The attribute should be applicable without changing the prim's type. Which schema approach is correct?

- ☐ A Create an `IsA` schema that derives from `UsdGeomMesh` to add the attribute to all meshes
- ☒ B Create a single-apply API schema that defines `simulation:restLength` and can be applied to any prim using `prepend apiSchemas`
- ☐ C Add the attribute directly to the USD core schema by modifying the USD source code

D

Store the attribute as USD metadata using `customLayerData` on the layer

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: A **single-apply API schema** is the correct choice for adding attributes to arbitrary prims without changing their type. API schemas are applied via `prepend apiSchemas = ["SimulationAPI"]` and contribute their defined attributes to any prim, regardless of its ISA type. This is how USD allows domain-specific data (physics, simulation, rendering hints) to be mixed into any prim.

#25

CUSTOMIZING USD · 6% EXAM WEIGHT



Two pipeline teams have created custom USD schema plugins that both define a schema named `MyAssetAPI`. What problem does this cause, and how should it be prevented?

A

No problem — USD allows multiple schemas with the same name as long as they are in different libraries

B

Schema name collision — USD's plugin registry will experience undefined behavior when both plugins are loaded; prevent this by using unique, studio-prefixed names for all custom schemas

C

The second plugin to load will automatically replace the first plugin's schema definition

D

USD raises a compilation error when building the second plugin

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: **Schema name collisions** are a real risk in USD pipelines. If two plugins register a schema with the same name, USD's type registry has undefined behavior — one may silently override the other, or both fail to register correctly. The prevention is to use **unique, studio-prefixed names** (e.g., `StudioXSimulationAPI`, `PixarRenderPassAPI`). This is why NVIDIA, Pixar, and other organizations use distinct prefixes.

#26

CUSTOMIZING USD · 6% EXAM WEIGHT



A developer is choosing between writing a `UsdSchemaBase`-derived plugin vs. a `SdfFileFormat` plugin to support a new proprietary interchange format.

The format stores scene data similarly to USD but uses a binary encoding. Which plugin type is appropriate?

- A A ``UsdSchemaBase`` plugin — it handles all binary format reading
- B A ``SdfFileFormat`` plugin — it allows the proprietary format to be opened as a native USD layer, with on-the-fly translation to USD scene description**
- C A ``HdRenderDelegate`` plugin — binary formats require a custom renderer to decode them
- D An ``ArResolver`` plugin — it handles the binary path resolution needed to open proprietary files

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: ``SdfFileFormat`` is the plugin interface for reading (and optionally writing) non-native file formats as USD layers. By implementing ``SdfFileFormat``, the proprietary binary format can be opened by USD as if it were a standard USD layer — USD's composition system reads it transparently. The plugin translates the binary format to ``SdfAbstractData`` on the fly.

#27

CUSTOMIZING USD • 6% EXAM WEIGHT



A studio wants to add variant fallback selections to their pipeline so that if a variant named ``rendering`` has no explicit selection, it defaults to ``final`` instead of the first defined variant. Where is this configured?

- A In the asset's USD file by setting ``variantSelections = {string rendering = "final"}`` at the top level
- B In the stage's root layer metadata using ``variantSets`` entries to declare defaults
- C Through the ``PcpVariantFallbacks`` mechanism, configured in a plug-in's ``plugInfo.json`` or set programmatically on the stage via the `UsdStage` API**
- D By setting ``defaultVariant = "final"`` in the schema definition for the variant set

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Variant fallbacks in USD are configured via the `PcpVariantFallbacks` mechanism. They can be set: (1) programmatically using `UsdStage::SetGlobalVariantFallbacks()`, (2) via a plugin's `pluginInfo.json` that registers fallback selections, or (3) through studio pipeline configuration that the stage picks up on initialization. This allows pipeline-wide defaults for variant selections without modifying asset files.

DATA EXCHANGE

#28

DATA EXCHANGE • 15% EXAM WEIGHT

A USD-to-gITF converter encounters a prim with `UsdGeomMesh` that has `subdivisionScheme = catmullClark`. gITF does not support subdivision surfaces. What should the converter do?

- A Skip the prim entirely — converters should only export data with direct gITF equivalents
- B Tessellate the subdivision surface to a polygon mesh before export, since gITF only supports polygon geometry**
- C Export the `catmullClark` value as a gITF extension attribute and let the gITF viewer handle it
- D Convert the prim to a `UsdGeomNurbsPatch` before the gITF export

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Since gITF only supports polygon meshes, a USD-to-gITF converter must **tessellate subdivision surfaces to polygons** before export. This is a *lossy conversion* — the parametric subdivision description is lost and replaced by a static polygon approximation. The resolution of tessellation (how fine the polygon approximation is) is a converter parameter. This trade-off should be documented in the conceptual data mapping.

#29

DATA EXCHANGE • 15% EXAM WEIGHT

A pipeline uses `usdzip` to package a USD asset for delivery. Which statement correctly describes what `usdzip` does?

- A `usdzip` compresses USD layers using gzip for smaller file sizes

B `usdzip` creates a USDZ archive by packaging the root USD layer and all of its dependencies (referenced layers, textures, etc.) into a single uncompressed zip file

C `usdzip` converts USDA files to USDC format and stores them in a zip archive

D `usdzip` is a validation tool that checks whether all referenced files exist before delivery

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: `usdzip` packages a USD scene and all its dependencies into a **USDZ archive** — an uncompressed zip file (uncompressed so that assets inside can be memory-mapped directly). The tool traverses the USD stage, collects all referenced layers and assets (textures, materials, etc.), and bundles them into the USDZ. The archive is self-contained and can be shared without separate asset directories.

#30

DATA EXCHANGE • 15% EXAM WEIGHT

An importer receives a mesh where the source DCC uses a left-handed coordinate system (X right, Y up, Z forward into screen). USD conventionally uses right-handed coordinates (X right, Y up, Z toward viewer). How should the importer handle this?

A Negate all Z-position values and reverse face winding order to convert from left-handed to right-handed

B Store the geometry as-is and set a custom metadata attribute `coordinateSystem = left-handed` to inform downstream tools

C Only negate Z positions — winding order does not need to change between coordinate systems

D USD automatically detects coordinate handedness and applies a correction at render time

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Converting from left-handed to right-handed coordinates requires **negating one axis (Z) and reversing face winding order**. Negating Z flips the handedness, but alone this also mirrors the geometry which inverts face normals (by reversing winding). To restore correct

normal orientation, winding order must be reversed too. USD does not auto-convert handedness — the importer must handle this explicitly.

#31

DATA EXCHANGE • 15% EXAM WEIGHT

A DCC application natively stores its scene hierarchy as a graph (nodes with multiple parents). USD's scene description uses a strict tree hierarchy (each prim has exactly one parent). How should a DCC exporter handle shared nodes?

- A Export only one copy of shared nodes — arbitrarily pick one parent path
- B Use USD native instancing: define the shared node content once and reference it from multiple parent locations using instanceable prims**
- C Create a flat list of all nodes and ignore the hierarchy entirely
- D Store the multi-parent relationship as USD metadata and let importers reconstruct the graph

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: USD's strict tree hierarchy cannot directly represent graph (multi-parent) relationships. The best approximation is **USD native instancing**: define the shared node's content as a prototype and reference it from each parent location with ``instanceable = true``. Each reference creates a separate prim in the hierarchy but they share the prototype's data. This is the standard USD pattern for DCC-style instanced nodes.

#32

DATA EXCHANGE • 15% EXAM WEIGHT

A 'round-trip' pipeline is tested by exporting a USD file from DCC A, importing it into DCC B, and re-exporting back to USD. The final USD file is compared to the original. Which type of discrepancy indicates a genuine data fidelity issue (not just a formatting difference)?

- A The re-exported file uses USDC format while the original was USDA
- B The re-exported file has different whitespace and comment formatting in the USDA text

C Vertex positions have changed by 0.001 units due to floating-point precision loss during import/export

D The re-exported file has updated timestamps in the file header

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: **Changes to vertex positions** (or any actual data values) indicate a genuine data fidelity issue — floating-point precision loss during the import/export round-trip. Format differences (USDA vs USDC), whitespace, comments, and timestamps are cosmetic. A round-trip pipeline must minimize or eliminate numerical precision loss to be considered faithful.

#33

DATA EXCHANGE • 15% EXAM WEIGHT

A USD exporter needs to write material assignments where the same mesh uses different materials on different face groups. After export, the validator reports `UsdGeomSubset` prims are missing. What is the consequence for downstream rendering?

A The renderer will crash because material binding without subsets is illegal in USD

B Only the material bound to the Mesh prim itself will be used — all face-group materials will be lost, and the mesh will render with a single material

C The renderer will infer face group materials from the `faceVertexCounts` array

D Downstream tools will automatically recreate the subsets from the material assignment metadata

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Without `UsdGeomSubset` prims, there is no way to assign materials to specific face groups — USD only knows about the single material bound to the Mesh prim. The renderer will use **only that single mesh-level material for the entire mesh**, losing all per-face material variation. This is a common and impactful DCC export bug for multi-material meshes.

#34

DATA EXCHANGE • 15% EXAM WEIGHT

A pipeline team is documenting the data mapping between their proprietary material system and `UsdPreviewSurface`. Their proprietary system has a `retroreflection` property with no USD equivalent. How should this be documented in the mapping?

- ☐ A Mark it as 'Not Supported' and exclude it from import/export entirely
- ☒ B Document it as 'No Direct Equivalent' — store in `userProperties:mat:retroreflection` as a custom float attribute; omit from USD Preview Surface shading but preserve for future schema extension or proprietary renderer support
- ☐ C Map it to `UsdPreviewSurface.roughness` as the closest approximation
- ☐ D Document it as requiring a complete custom schema before any export can proceed

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Best practice for unmapped properties is to **preserve them as custom attributes** in a documented namespace (`userProperties:` or a studio-specific namespace) so the data survives round-trips. The mapping document should explicitly state: what the property is, that it has no direct USD equivalent, where it is stored, and what its status is (e.g., 'not rendered in UsdPreviewSurface but available for proprietary renderers'). This maintains data fidelity without blocking the export.

#35

DATA EXCHANGE • 15% EXAM WEIGHT

A pipeline needs to determine whether an existing `.usd` file is in ASCII (USDA) or binary crate (USDC) format without loading it as a USD stage. What is the most reliable method?

- ☐ A Check if the file size is under 1MB — USDA files are always smaller
- ☒ B Inspect the first few bytes of the file: USDC files begin with `PXR-USDC`, USDA files begin with `#usda`
- ☐ C Check the file's creation timestamp — USDC is always newer than USDA
- ☐ D Run `usdchecker` on the file and read the format from its output

[CHECK ANSWER](#)[SHOW EXPLANATION](#)[FLAG](#)

Explanation: The most reliable method is to inspect the **magic bytes** at the start of the file: USDC binary crate files begin with `PXR-USDC`, and USDA ASCII files begin with `#usda`. This works for any `.usd`, `.usda`, `.usdc`, or even `.usdz` file (for embedded layers) without needing to open a full USD stage. File size and timestamps are unreliable indicators.

#36

DATA EXCHANGE • 15% EXAM WEIGHT

A USD asset validator checks that all prims using `UsdShadeMaterialBindingAPI` have a valid material target. A prim has `material:binding = ` but there is no prim at that path. What should the validator report?

A

No error — dangling relationship targets are valid in USD and resolve to no-ops

B

A warning that the material relationship is dangling — the target path does not exist in the composed stage, meaning the mesh will render without the intended material

C

A fatal error that prevents the stage from opening

D

An info message noting that the material will be loaded from the default texture library

[CHECK ANSWER](#)[SHOW EXPLANATION](#)[FLAG](#)

Explanation: A **dangling relationship target** (a relationship pointing to a non-existent prim) is technically valid USD (it does not cause a composition error), but it represents a data integrity problem. A validator should report this as a **warning or error** because it means the mesh has no material at render time. `usdchecker` includes checks for dangling material bindings as part of its asset compliance rules.

#37

DATA EXCHANGE • 15% EXAM WEIGHT

A studio is exporting camera data from their proprietary camera system to USD. Their cameras store focal length in pixels; USD stores it in world-space units (typically millimeters). What must the exporter do?

A Store the pixel value directly — USD automatically converts pixel focal lengths to millimeters

B Convert the focal length from pixels to the appropriate world-space unit using the sensor/aperture dimensions, and store the result in the `focalLength` attribute on `UsdGeomCamera`

C Store the pixel value in a `userProperties:focalLengthPixels` attribute and let USD compute the millimeter value

D Omit the focal length — USD cameras do not require focal length to render correctly

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: USD camera attributes (`focalLength`, `horizontalAperture`, etc.) use world-space units (typically millimeters). The exporter must **convert the proprietary pixel focal length to millimeters** using the sensor dimensions ($\text{focal_length_mm} = \text{focal_length_px} \times \text{sensor_width_mm} / \text{image_width_px}$). Storing raw pixel values in USD camera attributes will produce incorrect renders because renderers interpret these values as millimeters.

#38

DATA EXCHANGE • 15% EXAM WEIGHT

A pipeline imports animation from Alembic files into USD. The Alembic file contains animated geometry caches. What is the most appropriate USD representation for this data?

A Convert each Alembic frame to a separate USD variant and animate between variants

B Use USD value clips — map the Alembic file's time samples to USD clip layers, allowing USD to read Alembic time samples through a file format plugin

C Bake all Alembic animation frames into a single USDC file with USD time samples on the `points` attribute

D Store Alembic animation as string attributes containing JSON-encoded point arrays

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: The most direct and performant approach is to **bake the Alembic time-sampled geometry into USD time samples** on attributes like ``points``, ``normals``, and ``velocities``. This produces standard USD geometry animation that any renderer can use. Alternatively, a file format plugin can read Alembic files as USD layers directly, but for robust pipeline compatibility, baking to explicit time samples in USDC format is preferred.

#39

DATA MODELING · 15% EXAM WEIGHT

A prim has a ``half`` typed attribute storing a floating-point value. What is the key trade-off of using ``half`` versus ``float`` for this attribute?

A ``half`` is a 16-bit float with reduced precision range compared to 32-bit ``float`` — appropriate for color data or normalized values where extreme precision is not needed, and uses half the storage

B ``half`` stores integers; ``float`` stores real numbers

C ``half`` is only supported in USDC format; ``float`` works in both USDA and USDC

D ``half`` is automatically interpolated between time samples; ``float`` is held

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: ``half`` is USD's 16-bit floating-point type (IEEE 754 half-precision). It has less precision and a smaller range than 32-bit ``float``, but uses half the memory and I/O bandwidth. It is appropriate for data like HDR color components, normalized weights, and UV coordinates where high precision is less critical. Using ``half`` for position data is risky due to precision loss at large coordinate values.

#40

DATA MODELING · 15% EXAM WEIGHT

A mesh has an attribute ``int[] faceVertexCounts = [4, 4, 3, 3]``. What does this tell you about the mesh's topology?

A The mesh has 14 faces total ($4+4+3+3=14$)

B The mesh has 4 faces: two quads (4 vertices each) and two triangles (3 vertices each)

C The mesh has 4 vertices, distributed across 4 groups

D The mesh has 4 faces all with different winding orders

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: `faceVertexCounts` has one entry per face, specifying how many vertices define that face. `[4, 4, 3, 3]` means: 4 faces total — face 0 has 4 vertices (quad), face 1 has 4 vertices (quad), face 2 has 3 vertices (triangle), face 3 has 3 vertices (triangle). `faceVertexIndices` then contains $4+4+3+3 = 14$ entries total, indexing into the `points` array.

#41

DATA MODELING • 15% EXAM WEIGHT



A developer authors metadata on a prim using `prim.SetMetadata('myStudioTag', 'approved')`. Later, the stage is flattened. Does this metadata survive flattening?

A No — all custom metadata is stripped during flattening to produce a clean delivery layer

B Yes — prim metadata (registered or unregistered) is preserved during flattening as part of the prim spec

C Only metadata registered in a schema plugin survives flattening

D It depends on whether the metadata was authored in the root layer or a sublayer

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Prim **metadata survives flattening**. When `UsdStage::Flatten()` evaluates the stage, it collects all composed prim metadata (including custom/unregistered metadata) and writes it into the flattened layer. This is intentional — metadata is part of the prim spec and should be preserved for pipeline consumers of the flattened asset.

#42

DATA MODELING • 15% EXAM WEIGHT



Which USD data type is used to represent an ordered list of unique string tokens identifying applied API schemas on a prim?

A string[] — an array of schema name strings

B token[] — stored in the `apiSchemas` metadata field, where each token identifies an applied API schema

C asset[] — an array of asset paths pointing to schema definition files

D rel[] — a relationship array pointing to schema prim paths

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Applied API schemas are tracked in the prim's **`apiSchemas` metadata** as a **`token[]`** (token list-op). Each token is the name of an applied API schema (e.g., **`MaterialBindingAPI`**, **`GeomModelAPI`**). Using **`prepend apiSchemas = ["MaterialBindingAPI"]`** in USD is the way to apply an API schema to a prim. The list-op nature allows schemas to be prepended, appended, or deleted across layers.

#43

DATA MODELING • 15% EXAM WEIGHT



A UsdGeomMesh is authored without a `normals` attribute. How do Hydra-based renderers typically handle this?

A The mesh renders as completely flat with no lighting variation

B Renderers compute smooth normals from the mesh topology and vertex positions at render time

C The mesh is invisible until normals are explicitly authored

D USD automatically generates normals using the `catmullClark` subdivision algorithm

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: When no **`normals`** attribute is present on a **`UsdGeomMesh`**, Hydra-based renderers **compute smooth normals** from the mesh topology (vertex positions and face connectivity) at render time. This is the default behavior and produces correctly shaded results

for most meshes. Authored normals override this computation and are used for custom shading like hard edges.

#44

DATA MODELING • 15% EXAM WEIGHT

A ``UsdGeomXformable`` prim has ``xformOpOrder`` = `["xformOp:translate", "xformOp:rotateY", "xformOp:scale"]`. Which transformation is applied first to a vertex?

A Scale — listed last, but scale is always applied before rotation and translation

B Translate — listed first in `xformOpOrder`

C Scale — transform order is always TRS (translate, rotate, scale) in USD

D RotateY — rotation is always applied before translation in USD

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: In USD, ``xformOpOrder`` defines the order of matrix multiplication. The transforms are applied right-to-left when computing the final matrix — meaning the **first entry in the list is applied last** (outermost), and the last entry is applied first (innermost). However, from the perspective of a vertex being transformed: the rightmost op (scale) is applied first, then rotateY, then translate. But the question asks about `xformOpOrder` itself — 'translate' is listed first, meaning it wraps the others and is the last matrix multiplied in — effectively the first from the object's local space outward.

#45

DATA MODELING • 15% EXAM WEIGHT

A pipeline stores skeletal animation data in USD using ``UsdSkel``. Which prim type holds the skeleton topology (joint names and parent-child relationships)?

A `UsdGeomXform` — the skeleton is represented as a hierarchy of Xform prims

B `UsdSkelSkeleton` — a prim that defines the skeleton's joint names, rest transforms, and topology

C `UsdSkelRoot` — the root prim that marks the boundary of a `UsdSkel` rig

D

UsdSkelAnimation — the prim that stores joint transforms and blend shape weights

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: `UsdSkelSkeleton` defines the skeleton's *topology* — the ordered list of joint names (`joints` attribute), their rest-pose transforms (`restTransforms`), and bind transforms (`bindTransforms`). `UsdSkelRoot` marks the subtree as a skeletal rig. `UsdSkelAnimation` provides time-sampled joint transforms that drive the skeleton. `UsdSkelSkeleton` is specifically the structural definition of what the skeleton IS.

#46

DATA MODELING • 15% EXAM WEIGHT



A mesh attribute is declared as `uniform color3f[] primvars:perFaceColor`. The `uniform` qualifier has what effect on interpolation?

A

The attribute value is constant and cannot be time-sampled

B

The attribute stores one value per face (face-uniform interpolation) — the color is constant across each face but can differ between faces

C

The attribute value is shared uniformly across all prims in the stage

D

The `uniform` keyword forces the value to be authored in the root layer only

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: In USD primvar context, `uniform` interpolation means one value per face — the attribute stores one element for each face in the mesh. Within a face, the value is constant (no interpolation). Between faces, values can differ. This is commonly used for per-face material IDs, per-face display colors, or other per-face data. Note: in USD type system, `uniform` qualifier means 'not time-varying', but in primvar interpolation it means 'per-face'.

#47

DATA MODELING • 15% EXAM WEIGHT



A developer wants to store a collection of string labels associated with specific prims for a pipeline tagging system. Which USD construct is most appropriate?

- A ``string[]`` attribute on each prim containing the labels
- B A relationship on each prim pointing to other 'label' prims that represent each tag
- C Prim ``assetInfo`` metadata with a ``tags`` key containing a ``VtDictionary`` of label strings
- D** Custom metadata registered in a schema plugin as ``token[]`` on each prim (e.g., ``pipelineTags``)

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: For pipeline tagging, **custom metadata registered as ``token[]``** via a schema plugin is the most appropriate choice. It provides: type safety, schema discoverability, proper composition behavior, and efficiency (tokens are interned strings). ``assetInfo`` is available without plugins but is less structured. A ``string[]`` attribute works but lacks the semantic clarity of metadata. Relationships are for prim-to-prim structural links, not labels.

DEBUGGING

#48

DEBUGGING · 11% EXAM WEIGHT

A USD stage fails to open with the error: 'Cannot open layer @./missing_asset.usdc@'. The file exists on disk. What is the most likely cause?

- A The file is in USDC format — USD can only open USDA files as referenced layers
- B** The ``ArResolver`` cannot map the relative path to an absolute filesystem path — the working directory or asset context is incorrect for this stage
- C The file has a ``usdc`` extension but is actually USDA text
- D The stage was opened with ``UsdStage::CreateNew()`` instead of ``UsdStage::Open()``

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Relative paths like ``./missing_asset.usdc`` are resolved by the **ArResolver** relative to the anchoring layer's path. If the stage was opened with an incorrect working directory, if the

anchor path is wrong, or if the custom ArResolver has a configuration issue, the resolver may fail to map the relative path to the actual file location — even if the file exists. Checking the anchor layer's directory and resolver configuration is the first step.

#49

DEBUGGING · 11% EXAM WEIGHT

A USD tool reports that a prim's `kind` is `component`, but a pipeline tool that traverses the model hierarchy cannot find it. What is a likely reason?

A

The `component` kind is not supported by the default model hierarchy traversal in older USD versions

B

A parent prim in the hierarchy has `kind` unset or set to a non-model kind, breaking the model hierarchy — model hierarchy traversal requires an unbroken chain of model kind prims from the stage root

C

The `component` prim is using USDC format which model traversal tools cannot read

D

Model hierarchy traversal only finds prims with `kind = assembly`, not `component`

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: USD's model hierarchy traversal requires an **unbroken chain of model kind prims** from the stage pseudoroot down to the component. If any ancestor prim in the hierarchy has `kind` unset (or set to a non-model value), the traversal stops there and does not recurse to find the `component` below. This is a common setup error when building USD hierarchies — every prim in the path must have an appropriate model kind.

#50

DEBUGGING · 11% EXAM WEIGHT

An FX artist reports that a particle simulation looks correct in the first 100 frames but then shows unexpected 'ghost' positions starting at frame 101. The particles use value clips. What should you investigate?

A

The `faceVertexIndices` array likely has an off-by-one error starting at frame 101

B

The `clipActive` metadata — clip 0 may end at frame 100 and clip 1 may start at frame 101 with an incorrect time offset, causing positions to jump

C The particle system is using `half` precision for positions, causing overflow after frame 100

D The stage is configured with `UsdInterpolationType::Held` which causes ghost frames

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: The transition between value clips at frame 101 is the primary suspect. If **clipActive** switches from clip 0 to clip 1 at frame 101, but clip 1's `clipTimes` mapping is incorrect (wrong time offset or scale), positions in clip 1 will not correctly continue from where clip 0 left off — causing apparent 'ghost' positions or jumps. Inspecting the `clipTimes` and `clipActive` metadata for correct time mapping is the diagnostic step.

#51

DEBUGGING · 11% EXAM WEIGHT

A pipeline tool authoring opinions on a stage is accidentally writing to the wrong layer. What is the correct way to control which layer receives new authored opinions?

A Close and reopen the stage specifying the target layer as the root layer

B Use `UsdStage::SetEditTarget()` to redirect all subsequent authoring API calls to the desired layer

C Set the `active` metadata on the target layer before authoring

D Use `UsdStage::Save()` to first flush existing opinions before redirecting

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: `UsdStage::SetEditTarget()` changes the edit target to any layer in the stage's layer stack. All subsequent calls to USD's authoring APIs (attribute `Set()`, prim creation, variant selection, etc.) will write to the new edit target layer. This is how pipeline tools correctly route opinions to specific layers — e.g., writing shot overrides to the shot layer, not the root layer.

#52

DEBUGGING · 11% EXAM WEIGHT

A renderer reports that a mesh is not visible even though ``visibility = inherited`` is set on the mesh prim. What is the most likely explanation?

- ☐ A ``visibility = inherited`` always makes a prim invisible — you must use ``visibility = visible`` explicitly
- ☐ B An ancestor prim has ``visibility = invisible``, and ``inherited`` means 'inherit visibility from the parent' — the invisibility propagates down to the mesh
- ☐ C The mesh's ``purpose`` is set to ``guide``, which excludes it from rendering regardless of ``visibility``
- ☒ D Both B and C are valid explanations

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Both are valid: (B) ``visibility = inherited`` means the prim inherits its visibility from its closest ancestor with a non-``inherited`` visibility value. If any ancestor has ``visibility = invisible``, the invisibility propagates down. (C) A prim with ``purpose = guide`` is excluded from render traversal regardless of ``visibility``. Checking ancestor visibility values and the ``purpose`` attribute are both essential debugging steps for invisible geometry.

#53

DEBUGGING · 11% EXAM WEIGHT



A pipeline engineer runs ``usdchecker`` on a published asset and receives: `'Missing extent on /World/Body'`. The asset renders correctly in usdview. Why does this matter?

- ☐ A It doesn't — usdview proves the asset is correct; usdchecker rules are overly strict
- ☒ B Correct rendering does not require ``extent``, but many pipeline tools (bounding box queries, LOD selection, frustum culling) rely on it; a missing extent will cause incorrect behavior in those tools even if rendering looks fine
- ☐ C The ``extent`` attribute is only required for USDZ packaging — standard USD files can omit it
- ☐ D Missing ``extent`` causes geometry to be culled during flattening

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Rendering does not strictly require ``extent`` — renderers can compute bounds from geometry. However, many *pipeline tools* rely on ``extent`` for fast bounding box queries, LOD selection, frustum culling, camera framing, and scene layout — all without loading full geometry. A missing ``extent`` breaks these tools' ability to make correct decisions efficiently. ``usdchecker`` flags it as an error for good reason.

#54

DEBUGGING · 11% EXAM WEIGHT

A stage opens and a prim ``/World/Hero`` exists, but querying ``prim.GetAttribute('color3f[] primvars:displayColor').Get()`` returns ``None``. What are the possible reasons?

- ☐ A The attribute does not exist on the prim — ``Get()`` returns ``None`` for non-existent attributes
- ☐ B The attribute exists but has no authored value (neither a default nor a time sample at the queried time code) — ``Get()`` returns ``None`` when there is no value to return
- ☐ C The prim is inside a payload that has not been loaded — unloaded payload attributes return ``None``
- ☒ D All of the above are valid reasons for ``Get()`` returning ``None``

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: All three are valid causes of ``Get()`` returning ``None``: (A) the attribute simply doesn't exist; (B) the attribute exists (is on the prim spec) but has no authored default value or time sample at the queried time code; (C) the attribute's data is inside an unloaded payload — USD can tell the attribute exists (it may appear in ``GetAttributes()``) but cannot return a value without loading the payload.

#55

DEBUGGING · 11% EXAM WEIGHT

A USD stage has 10,000 prims and opening it takes 30 seconds. A profiling session shows most time is spent in composition (PrimCache Population). What changes would most improve opening performance?

- ☐ A Convert all USDA layers to USDC format

B Use payload arcs for heavy asset content so it is deferred; open the stage with ``UsdStage::Open(UsdStage::LoadNone)`` to skip payload loading until needed

C Reduce the number of variant sets in asset files

D Flatten the stage before opening it

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: If composition (PrimCache Population) is the bottleneck, the most impactful change is moving heavy asset content behind **payload arcs** and opening the stage with ``UsdStage::LoadNone`` (or an appropriate load set). This defers payload composition until those prims are explicitly loaded, dramatically reducing the number of composition operations needed to open the stage. Format conversion (USDA→USDC) helps I/O speed, not composition overhead.

PIPELINE DEVELOPMENT

#56

PIPELINE DEVELOPMENT • 14% EXAM WEIGHT

A pipeline tool needs to remove a reference that was added by a weaker asset layer from a shot stage, without modifying the asset file. Which USD operation accomplishes this?

A Set ``active = false`` on the referencing prim in the shot layer

B Use list editing ``delete`` to remove the specific reference arc in a stronger layer — the stronger layer's ``delete`` operation removes the reference from the composed reference list

C Replace the reference with an ``over`` prim that has no content

D Set ``visibility = invisible`` on the prim to prevent the reference from being rendered

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: USD's **list editing ``delete``** operation in a stronger layer can remove a reference (or payload, or inherit, or sublayer) that was added by a weaker layer — without modifying the

weaker layer. In the shot layer: ``delete references = [ @./unwanted_asset.usda@ ]`` removes that reference from the composed list. This is a key non-destructive pipeline operation.

#57

PIPELINE DEVELOPMENT · 14% EXAM WEIGHT

A production pipeline needs to track which USD layers have been modified since the last publish. Which approach is best suited for USD-based version control?

- A Store modification timestamps on each layer as custom metadata and compare them to a baseline
- B Flatten the entire stage before and after changes and compare the resulting files
- C Use standard version control systems (Git, Perforce) on individual USD layer files — since layers are discrete files, per-file VCS diffing and locking work naturally with USD's layer model**
- D Use USD's built-in change tracking API to generate a diff between two stages

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: USD's design of **discrete layer files** maps naturally to file-based version control systems like Git or Perforce. Each layer is a file that can be independently checked in, diffed, branched, and merged. USDA (ASCII) layers are especially VCS-friendly because they produce human-readable diffs. This is a core advantage of USD's multi-layer architecture for collaborative pipelines.

#58

PIPELINE DEVELOPMENT · 14% EXAM WEIGHT

A pipeline needs to build a USD stage that aggregates geometry, animation, lighting, and FX into a single render-ready stage. At which stage of the pipeline should the stage be flattened?

- A At the beginning, to create a simple base for all departments to work from
- B After each department adds their layer, to maintain a clean stage

C Only at the very end, for final render delivery or external handoff — flattening should be deferred as long as possible to preserve non-destructive editability throughout production

D Flattening should be done once per week as a checkpoint operation

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Flattening destroys the non-destructive layer structure. It should only be done at the **very end of the pipeline** — for final render delivery, archival, or external vendor handoff. During active production, maintaining the full layer stack is essential for: non-destructive overrides, per-department editing, version control per layer, and the ability to re-compose with updated assets. Early flattening is a pipeline anti-pattern.

#59

PIPELINE DEVELOPMENT • 14% EXAM WEIGHT

A USD asset has its geometry in USDA format and is 50MB. Converting it to USDC results in a 12MB file. What are the implications of this difference?

A The USDC file may have lost data — compression always implies data loss

B USDC stores the same scene description more compactly in binary form and supports memory-mapped access — the file is smaller and faster to read without any data loss; USDA's ASCII representation is inherently larger

C The USDC file is missing metadata because USDC does not support all USDA features

D The size difference indicates USDC uses lossy floating-point compression

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: USDC is a lossless binary format. The size difference between USDA (ASCII, ~50MB) and USDC (binary, ~12MB) is purely due to the encoding: ASCII text representation of numbers is much larger than binary encoding. No data is lost. USDC also supports memory-mapping for fast random access to time samples. Converting USDA → USDC is safe and recommended for production assets.

PIPELINE DEVELOPMENT • 14% EXAM WEIGHT

#60

A studio's pipeline has 5 departments each contributing a USD sublayer to a shot. A bug in one department's layer causes incorrect values on 100 prims. What is the fastest way to isolate the problematic layer without modifying any files?

- A Remove each sublayer from the list one by one, saving the root file each time
- B Use ``UsdStage::MuteLayer()`` to temporarily mute each department layer one at a time, recomposing the stage after each mute to see when the incorrect values disappear**
- C Create a new stage that only loads the suspect layer and check values there
- D Flatten all 5 layers and diff against the expected output

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: ``UsdStage::MuteLayer()`` is the ideal tool for non-destructive layer isolation. By muting one layer at a time and checking whether the incorrect values disappear, you can quickly binary-search to identify the problematic layer — without modifying any files on disk. When the muted layer causes the values to change back to expected, you've found the culprit.

#61

PIPELINE DEVELOPMENT • 14% EXAM WEIGHT

A character asset needs to support both a 'hero' (full detail, production quality) and a 'background' (reduced detail, optimized) configuration in the same USD file for a shot pipeline. What is the most flexible USD approach?

- A Create two separate USD files — one for hero and one for background
- B Use a ``detail`` variant set with ``hero`` and ``background`` variants that swap geometry, material, and LOD payload targets based on the selection
- C Use the ``purpose`` attribute — set production geometry to ``render`` and background geometry to ``proxy``
- D Both B and C are valid approaches and could be used together for maximum flexibility**

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Both approaches are valid and complementary: **Variant sets** allow switching entire configurations (geometry, materials, payload targets) between hero and background — controlled by pipeline or shot context. The **`purpose`** attribute separates render (full quality) from proxy (lightweight stand-in) geometry, letting viewers show the proxy while renderers use the full geometry. Using both together provides maximum flexibility in a production pipeline.

#62

PIPELINE DEVELOPMENT • 14% EXAM WEIGHT

A pipeline TD needs to ensure that all assets published to the library have their **`kind`** set correctly. What is the most scalable way to enforce this across hundreds of assets?

A

Train all artists on the model hierarchy and trust them to set **`kind`** correctly

B

Add a publish-time validation check that reads **`UsdModelAPI(prim).GetKind()`** on each asset root prim and rejects publishes where **`kind`** is unset or incorrect

C

Use a global USD plugin that automatically sets **`kind`** on all prims when a stage is opened

D

Store **`kind`** in a separate metadata file alongside each USD asset

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: The scalable approach is **automated publish-time validation**. A validation step that reads **`UsdModelAPI(prim).GetKind()`** on the asset root prim and enforces the correct kind value (based on asset type — component, assembly, etc.) creates a quality gate that cannot be bypassed. This scales across hundreds of assets without relying on per-artist discipline.

#63

PIPELINE DEVELOPMENT • 14% EXAM WEIGHT

A USD pipeline architect is designing the asset resolution strategy. They choose to use logical URIs (**`asset://studio/props/chair`**) over relative filesystem paths (**`./props/chair.usda`**). What is the primary advantage?

A

Logical URIs are natively supported by all DCC applications without custom plugins

B

Logical URIs decouple the asset address from the physical file location — assets can be moved, versioned, or mirrored on different storage systems without changing any USD

files; only the resolver's configuration changes

C Logical URIs are faster to resolve than relative paths because they bypass the filesystem

D Logical URIs are required for USDZ packaging to work correctly

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: The primary advantage of logical URIs is **location independence**. The USD files reference abstract addresses (``asset://studio/props/chair``) rather than physical paths. The ``ArResolver`` translates these to physical file locations. When assets are versioned, moved to different storage, or replicated to render farms, only the resolver's configuration changes — no USD files need updating. This is the foundation of scalable asset management.

#64

PIPELINE DEVELOPMENT • 14% EXAM WEIGHT

A studio decides to use ``UsdGeomXformCache`` when computing world transforms for many prims. Why is this preferable to calling ``UsdGeomXformable::GetLocalTransformation()`` for each prim individually?

A XformCache only works with USDC files — it requires binary format for caching

B XformCache computes and caches the full world-space transform (including all ancestor transforms) with time-coherent caching, avoiding repeated recomputation of ancestor transforms when traversing many prims at the same time code

C XformCache returns transforms in a different coordinate space than `GetLocalTransformation`

D XformCache is required for computing transforms on instanced prims

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: ``UsdGeomXformCache`` maintains a cache of computed transforms, avoiding redundant recomputation. When traversing many prims at the same time code, transforms that share ancestor prims can reuse cached ancestor transform computations. This is particularly valuable for large scenegraphs where repeated ``GetLocalTransformation`` calls on every prim

would redundantly recompute shared ancestor transforms, causing $O(\text{depth} \times \text{prims})$ work instead of $O(\text{prims})$.

#65

PIPELINE DEVELOPMENT • 14% EXAM WEIGHT

Which of the following best describes the principle of 'non-destructive editing' in a USD pipeline?

A

All USD edits are automatically saved to a backup file before being applied

B

Opinions are added in stronger layers that override weaker layers — the original data in weaker layers is never modified or deleted; the override can be removed by deleting the stronger layer's opinion

C

USD validates all edits before writing them to disk to prevent data corruption

D

Pipeline tools lock USD files during editing to prevent simultaneous modification

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: Non-destructive editing in USD means overrides are authored in stronger layers that win in composition, while the original weaker layers (the base assets) remain *untouched*. If the override needs to be removed, only the stronger layer's opinion is deleted — the original data is fully intact and immediately restored in composition. This is the fundamental design philosophy of USD's layering system.

VISUALIZATION

#66

VISUALIZATION • 8% EXAM WEIGHT

A `UsdShadeMaterial` prim contains two shader networks — one for Hydra Storm (using `UsdPreviewSurface`) and one for a production renderer (using a renderer-specific shader). How does a renderer know which network to use?

A

The renderer reads all shader networks and picks the one with the highest node count

B Materials expose multiple ``outputs:surface`` connections namespaced by render context (e.g., ``outputs:surface`` for universal, ``outputs:ri:surface`` for RenderMan) — renderers look for their specific context's output first, then fall back to the universal context

C The renderer uses the shader network that matches the ``metersPerUnit`` value in the layer

D The renderer automatically converts `UsdPreviewSurface` to its own shader language at load time

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: UsdShade uses **render contexts** (namespaced outputs) on the Material prim. For example: ``outputs:surface`` (universal/Storm), ``outputs:ri:surface`` (RenderMan), ``outputs:karma:surface`` (Karma). Each renderer looks for its specific context output first and falls back to the universal context. This allows a single material to contain multiple renderer-specific networks, enabling production renderers to use their native shaders while Hydra Storm uses `UsdPreviewSurface`.

#67

VISUALIZATION • 8% EXAM WEIGHT



A mesh prim has ``primvars:displayColor = [(1, 0, 0)]`` (red) with ``constant`` interpolation. A material with a blue ``diffuseColor`` is bound to the same prim. When rendered with Hydra Storm, which color appears?

A Red — ``primvars:displayColor`` always overrides bound material colors

B Blue — the bound material's ``diffuseColor`` takes precedence over display primvars when a material is bound

C Purple — Hydra Storm blends the two colors

D The behavior is undefined — you cannot have both a material and `displayColor` on the same prim

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: When a **bound material** exists, Hydra Storm uses the material's shader network for rendering. ``primvars:displayColor`` is a display hint used as a *fallback* when no material is bound — it is not a shader input that overrides material shading. The bound material's ``diffuseColor`` =

blue` takes precedence. `primvars:displayColor` is only used when no material is present (useful for quick viewport visualization without materials).

#68

VISUALIZATION · 8% EXAM WEIGHT

A `UsdGeomCamera` in USD stores its aperture values. What is the relationship between `horizontalAperture`, `focalLength`, and the camera's field of view?

A

Field of view = horizontalAperture × focalLength

B

Field of view is computed as: $2 \times \arctan(\text{horizontalAperture} / (2 \times \text{focalLength}))$ — a larger aperture or shorter focal length gives a wider field of view

C

The field of view is stored directly on the camera as a `float` attribute and aperture/focalLength are only for depth-of-field calculations

D

Field of view is fixed at 60 degrees in USD regardless of aperture and focal length

CHECK ANSWER

SHOW EXPLANATION

FLAG

Explanation: The **field of view** of a physical camera is derived from the relationship between *sensor size* (horizontal aperture) and *focal length*: $\text{FOV} = 2 \times \arctan(\text{aperture} / (2 \times \text{focalLength}))$. USD stores the physical camera parameters (`horizontalAperture` in mm, `focalLength` in mm) rather than derived FOV, matching how real cameras work. Renderers compute the actual FOV from these parameters.

#69

VISUALIZATION · 8% EXAM WEIGHT

An artist wants to limit a UsdLux `SphereLight` to only illuminate a specific subset of objects in the scene, without using render layers. Which UsdLux feature enables this?

A

Set the light's `intensity` to 0 on all prims except the intended targets

B

Use UsdCollectionAPI on the light to define a `lightLink` collection that includes only the prims the light should illuminate

C

Set `purpose = render` on the target prims and `purpose = proxy` on all other prims

D

Use the light's `radius` attribute to limit the physical range of illumination

[CHECK ANSWER](#)[SHOW EXPLANATION](#)[FLAG](#)

Explanation: **Light linking** in UsdLux uses **`UsdCollectionAPI`** applied to the light prim. A **`lightLink`** collection defines which prims a light illuminates (include/exclude lists). A **`shadowLink`** collection controls which prims cast shadows from that light. This allows fine-grained control over light-geometry interactions without render layers or geometry modifications.

#70

VISUALIZATION • 8% EXAM WEIGHT



A rendering engineer observes that a geometry-heavy USD scene with correct materials renders at an acceptable speed, but the same scene with a large number of UsdLux lights is very slow. What USD/Hydra feature can help manage this?

A

Convert all lights from UsdLux to custom shader-based emissive materials

B

Use ``UsdLuxLightAPI.inputs:enableColorTemperature`` to reduce light computation overhead

C

Place all lights behind payload arcs and only load the lights needed for each shot using the stage's load set — reducing the number of active lights Hydra processes

D

Set ``purpose = proxy`` on all lights not actively contributing to the scene

[CHECK ANSWER](#)[SHOW EXPLANATION](#)[FLAG](#)

Explanation: Placing lights behind **payload arcs** and using USD's **working set management** (``UsdStage::LoadSet``) to load only the lights needed for a specific shot or render region is an effective strategy for managing large numbers of lights. Unloaded payload lights are not evaluated or sent to Hydra, reducing the renderer's light count to only what is needed. This is the USD-native approach to shot-level light management.